

Compiler

Introduction

Jyun-Ao Lin

iFIRST & CSIE, TAIPEI TECH

Table of contents

1. INTRODUCTION

Introduction

Credits

A large part of this course is based on the Compilation Course of J.-C. Filliâtre at ENS Ulm.

Course organization

- **Course** Wednesday 13:10–14:00, Friday 10:10–12:00
- **Location** 宏裕 334(e)
- **Contact** jalin@ntut.edu.tw, 宏裕科技大樓 308
- **course materials** on Teams
- **homework submission** over 北科i學園

Evaluation

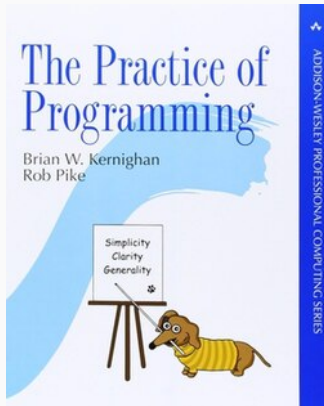
- 4 programming assignments: 30%
(we will program in OCAML <https://ocaml.org>)
 - (at least) one by each group (≈ 10 people)
 - and present in the class (Wednesday)
 - will be evaluated by other groups and also the instructor (equal weights)
 - and also by the group members themselves: with contribution
 - hi: 100%, mid: 90%, low: 80%, no: 60%

$$\forall S \in \text{Grp}_i, \text{Grades}_S = \frac{1}{5} \left(\sum_{j \neq i} G_j + \text{Instructor} \right) \times \text{contribution}_S$$

There is no such thing as a best programming language,
only good programmers.

Suggestion

The Practice of Programming by Brian W. Kernighan and Rob Pike



Evaluation

- a final **project** = a mini GO compiler: 40%
 - along or in pair
 - however, if you wish, you can use whatever programming language you like
 - extra points if you have verified version in ROCQ or LEAN
- midterm exam 30%
 - handwritten exam, open book, no internet, no AI, etc

Tentative schedule

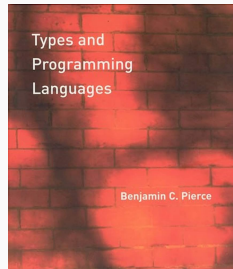
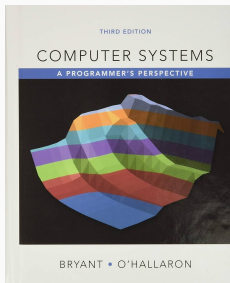
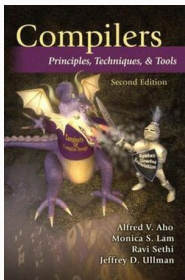
Week	Date	Topic
01	09/09, 11	Course info, OCaml intro
02	09/16, 18	x86-64 Assembly
03	09/23	AST
03	09/25	Semantics
04	09/30	HW1
04	10/01	Interpreter
05	10/07, 09	no class: OOPSLA
06	10/14, 16	Lexical analysis
07	10/21	HW2: mini-Python interpreter
07	10/23	Parsing
08	10/28, 30	Parsing
09	11/04, 06	Typing
10	11/11	HW3: mini-Turtle
10	11/13	Mid-term

Tentative schedule

Week	Date	Topic
11	11/18, 20	Parameters passing
12	11/25	Compiler correctness
12	11/27	Compilation of FP
13	12/02	HW4: W algorithm
13	12/04	Compilation of FP
14	12/09, 11	Compilation of OO
15	12/16, 18	Compiler optimization
16	12/23	Compiler optimization
16	12/25	no class: Noël
17	12/30	GC
17	01/01	no class: Happy New Year
18	01/06, 08	GC

Textbooks

- No textbooks but some useful references
 - [ALSU06] Alfred V. Aho, Monica S. Lam, Ravi Sethi et Jeffrey D. Ullman, *Modern Compilers: Principles, Techniques, and Tools*, the Dragon Book.
 - [BO16] Randal E. Bryant et David R. O'Hallaron, *Computer Systems: A Programmer's Perspective*.
 - [Pie02] Benjamin C. Pierce, *Types and Programming Languages*.
 - [Ler06], Xavier Leroy, Formal certification of a compiler back-end, or: programming a compiler with a proof assistant



AI

- There are now *many* tools that combine AI, coding and reasoning!
- **But!** you need to be able to understand AI-generated content and:
 - when its suggestions are good to use
 - when it's better to take its suggestions and modify them before use
 - when it's better to completely discard them
- For introductory class, the **process** is more important than the end result

No laptops or phones during lecture

- Laptops *closed* . . . minds *open*
- Why?
 - Strong temptation to surf/chat/email/game/text/tweet/etc.
 - This distracts not only you, but those around you
 - Better to take notes **by hand**
 - You will get plenty of time in front of your computer while working on the homework :-)

Course objectives

Understand the mechanisms behind **compilation**,
that is, the translation from one language to another.

Understand the various aspects of **programming languages**,
via their compilation.

The need of new compilers

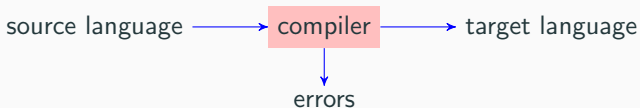
- New kinds of optimizations for new kinds of hardware (FPGA, RISC-V, etc) and new kinds of domain-specific semantics (e.g. R, TENSORFLOW, ..etc)
 - $O(m \times n)$:
 - $m = \#$ of programming languages
 - $n = \#$ of target hardware
- No one knows how to write compilers to unlock the full potentials of those domain-specific hardware¹

In other words, the science of compilers is not advanced yet and has not caught up with the massive explosion of domain-specific languages and novel hardware accelerators

¹c.f. [MDHS09] for some interesting facts

Compilation

A compiler translate a “program” from a **source** language to a **target** language, possibly signaling errors.



Detecting (**static**) errors like

- malformed identifiers, unclosed comments ...
- incorrect syntactic constructions
- undeclared identifiers
- mistyped expressions, e.g. `if 3 then "toto" else 4.5`
- uninstantiated references
- etc

Compilation to machine language

Compilation typically evokes translating a high-level language (C, JAVA, OCAML, etc.) to some machine language

```
% gcc -no-pie -o sum sum.c
```

source sum.c $\longrightarrow$ C compiler (gcc) $\longrightarrow$ executable sum

```
int main(int argc, char **argv) {
    int i, s = 0;
    for (i = 0; i <= 100; i++) s += i*i;
    printf("0*0+...+100*100 = %d\n", s);
}
```

$\longrightarrow$

```
001001111011110111111111111100000
101011111011111110000000000010100
10101111101001000000000000100000
10101111101001010000000000100100
1010111110100000000000000011000
1010111110100000000000000011100
1000111110101110000000000011100
...
```

Target language

In this course, we are going to consider compiling to **assembly**, indeed, but this is just only one aspect of compilation

Many techniques used in compilers are not related to the production of assembly code.

Moreover, some languages are instead

- interpreted (BASIC, COBOL, RUBY, PYTHON, etc.)
- compiled into some intermediate language, which is then interpreted (JAVA, PYTHON, OCAML, SCALA, etc.)
- just-in-time (on-the-fly) compiled (JULIA, etc.)
- compiled into another high-level language (LEAN, etc)

Differences between a compiler and an interpreter

- A **compiler** translates a program P into a program Q such that for any input x , the output of $Q(x)$ is identical to that of $P(x)$

$$\forall P \exists Q \forall x \dots$$

- An **interpreter** is a program that, given a program P and some input x , computes the output s of $P(x)$

$$\forall P \forall x \exists s \dots$$

Differences between a compiler and an interpreter

In other words

- the compiler performs a more complex task **only once**, to produce a code that accepts any input
- the interpreter performs a simpler task, but repeats it for every input

Another difference: compiled code is typically more efficient than interpreted code

- best interpreters are at least 10x slower than compiled code
(e.g. PYTHON ~30–50x)
⇒ use up > 10x more energy
⇒ Facebook compiles PHP to C++

Example of compilation and interpretation²

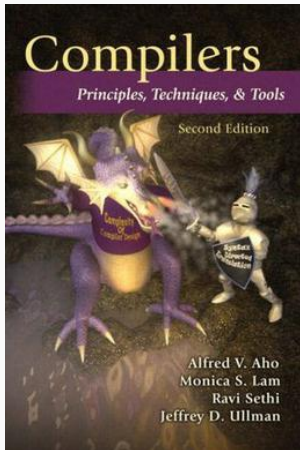
source → lilypond → PDF file → evince → image

```
\new PianoStaff <<
  \new Staff { \clef "treble" \key d \major \time 3/8
    <<d8. fis,8.>> <<cis'8. e,8.>> | ... }
  \new Staff { \clef "bass" \key d \major
    fis,,4. ~ | fis4. | \time 4/4 d2 }
>>
```



²Adopted from J.-C. Filliâtre

Quality of a compiler



How can we evaluate the quality of a compiler^a?

- its soundness (i.e. correctness)
- the performance of the compiled code
- the performance of the compiler itself

“Optimizing compilers are so difficult to get right that we dare say that no optimizing compiler is completely error-free! Thus, the most important objective in writing a compiler is that it is correct.”^b

(Dragon Book [ALSU06]).

^ac.f. SIGPLAN Empirical Evaluation Guidelines

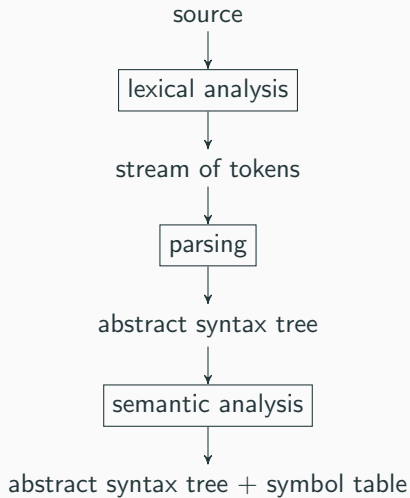
^bNow we have at least a verified compiler COMPCERT C for C language by Xavier Leroy [Ler06]: the test generation tool CSMITH [YCER11] found 79 bugs in GCC and 202 bugs in LLVM but was unable to find any bugs in the verified parts of COMPCERT.

Compiler phases

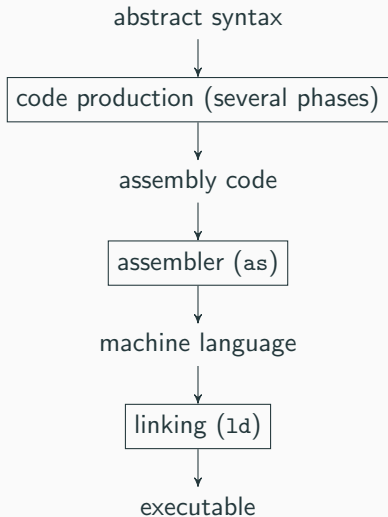
Typically, the compiler decomposes into

- a **front-end/analysis phase**
 - recognizes the program and its meaning
 - signals errors and thus can fail
(syntax errors, scoping errors, typing errors, etc.)
- and a **back-end/synthesis phase**
 - produces the target code
 - uses many intermediate languages
 - sometimes optimizes
 - must not fail

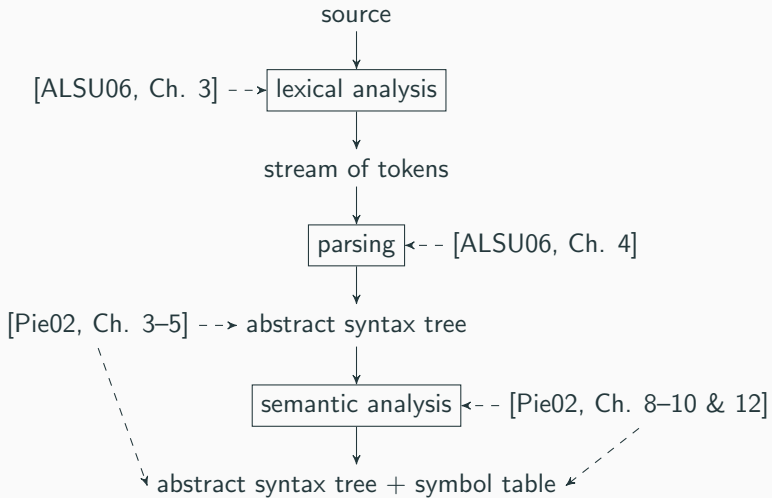
Standard front-end



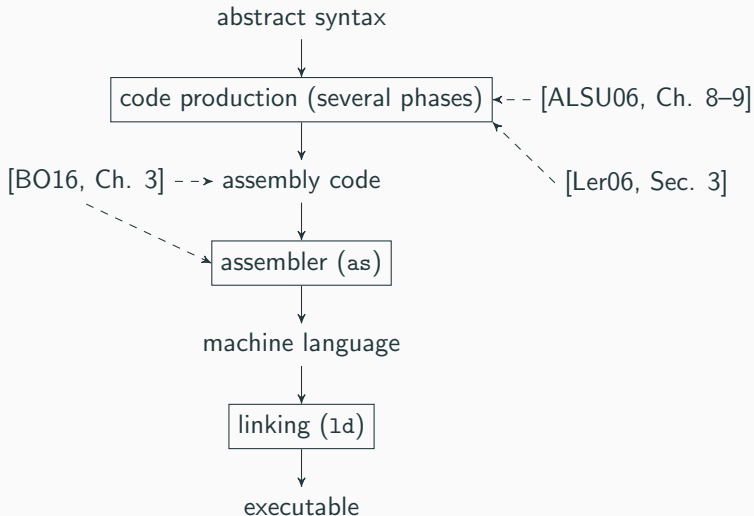
Standard back-end



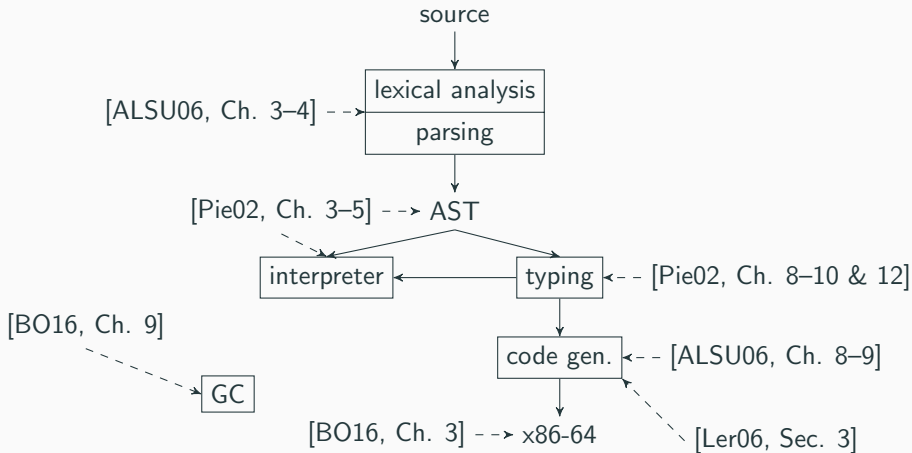
Standard front-end



Standard back-end



Overview of the course



References i



Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman.
Compilers: Principles, Techniques, and Tools (2nd Edition).
Addison-Wesley Longman Publishing Co., Inc., USA, 2006.



R.E. Bryant and D.R. O'Hallaron.
Computer Systems: A Programmer's Perspective.
Always Learning. Pearson, 2016.



Xavier Leroy.
Formal certification of a compiler back-end, or: programming a compiler with a proof assistant.
In 33rd ACM symposium on Principles of Programming Languages, pages 42–54. ACM Press, 2006.

References ii



Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney.

Producing wrong data without doing anything obviously wrong!

In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS XIV, page 265–276, New York, NY, USA, 2009. Association for Computing Machinery.



Benjamin C. Pierce.

Types and Programming Languages.

The MIT Press, 1st edition, 2002.



Xuejun Yang, Yang Chen, Eric Eide, and John Regehr.

Finding and understanding bugs in c compilers.

SIGPLAN Not., 46(6):283–294, jun 2011.

Questions?